## Student Declaration of Academic Integrity

By submitting this assignment you confirm that you have read, understood and abided by the following statements

1. You understand the policies on plagiarism of both Dickson College and the Board of Senior Secondary Studies;
2. The work you have submitted is your own work, it has not been substantially generated or plagiarised from generative AI or another source, or substantially edited and/or rewritten by other accounts, and it has not been submitted for assessment before;
3. You have planned and completed the assessment only in the Google Document assigned to you in the Google Classroom [or other files, as directed by your teacher and task document];
4. You understand that work created in any other document or file outside of your teacher's instructions may not be considered or marked;
5. You have appropriately referenced all sources of information that are not your own, including the words, images, and ideas of others;
6. You have included appendices for all images used when applicable;
7. Your work and processes are in line with the task requirements set out in this document;
8. You are aware that your assignment will automatically go through a plagiarism check via Google Classroom

# Digital Solutions Prototype

| | |
|---|---|
| Problem-solving Log | Completed |
| Log 1 | Completed |
| Log 2 | Completed |
| Log 3 | Completed |
| Bibliography (brief explanation of each link plz) | Completed |
| Demonstration Video | Completed |
| Code | Completed |
| Interview Notes | |

# Problem-solving Log (1500-2500 words T) (5 Pages A)

## Log 1

### Feature definition

Implement CSV import to allow django to convert and display the data through HTML. This is important because it helps make the website look clean and readable.

### Solution process

I started by creating a simple database and an admin portal. Using this setup, I was able to store teacher information and display details such as the teacher's name and age on the Django website. One of the first challenges I faced was understanding what a CSV file actually is and how it works. In simple terms, I learned that a CSV file converts plain text into a structured data format that can easily be read and processed by programs such as Python.

Using the following code,

```python
import csv

from django.core.management import BaseCommand
from MyApp1.models import teacher

class Command(BaseCommand):
    help = "This is some help text"
    def add_arguments(self, parser):
        parser.add_argument('--path', type=str)
    def handle(self, *args, **kwargs):
        path = kwargs['path']
        with open(path, 'rt', encoding='utf-8-sig') as f:
            reader = csv.reader(f, dialect='excel')
            for row in reader:
                teacher.objects.create(Name=row[0], Area=row[1])

```

I

was able to import teacher data from a CSV file into the Django database. Once the data was stored in the database, it could then be displayed on the website through HTML templates.

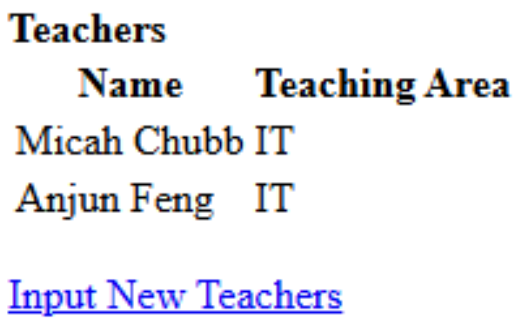
Another challenge I encountered was linking HTML with Django. This process was slightly complex because multiple Python files need to work together for the system to function correctly. In standard HTML development, it is easy to link files using a single line of code, such as `<link rel="stylesheet" href="style.css">`. However, Django requires additional configuration to properly connect HTML pages to the backend logic.

To begin, I created an HTML file called index.html inside the **templates** folder. After that, I created a view inside the views.py file. In this file, I used the line `from django.shortcuts import render`. The `render` function tells Django to send the HTML page to the browser so it can be displayed and manipulated through HTML.

Next, I linked the `Index.html` file to the view. However, I initially missed an important step: connecting the view to the URL configuration. This step is necessary because it tells Django which Python code should run when a user visits a specific page. For example, the following line of code tells Django that when a user visits the Index page, the `Index` function should run:

```
re_path(r'^$', MyApp1.views.index, name='index')
```

Once this step was completed, the HTML page was successfully connected to Django, allowing the website to display and interact with the data stored in the database.



## What have I learned?

During this process, I learned how to implement CSV files and link HTML to Django using the urls.py and views.py pages. I also learned how to use Django template commands to combine Python with HTML. This is really useful for future projects because it makes building websites much easier and more efficient.

Working with CSV files gives a good starting point for future projects since it allows data to be imported and used inside a Django application or other Python programs. This makes it much easier to manage information and display it on a website.

Linking HTML to Django is also an important skill because it helps you understand how the different parts of a web application connect, especially through the urls.py and urls.py pages. Using HTML to design the webpage also gives more flexibility and control over how the site looks, compared to doing everything directly in Django, which can be more time-consuming.

In the future, if I want to add more pages to my website, I'll be able to navigate the project structure more easily and connect new pages to Django without too much trouble. This makes it easier to expand and improve the website as the project grows.

# Log 2

## Feature definition

Creating a form that adds a teacher and their subject area. This is an important feature because it lets the user add or update the teachers database.

## Solution process

I wanted to add a form that allows users to update or add new teachers to the database, which would then be displayed on the website. This process is relatively straightforward: you create a new HTML page, a view in Django, and then display the data from the database.

In the views.py, I started by creating a view function called `def input_view(request)`. This function runs whenever a user visits the page connected to it. Inside this function, I used `request.POST` to check how the page was accessed. Essentially, this means that when the user submits the form, Django will run the code below it:

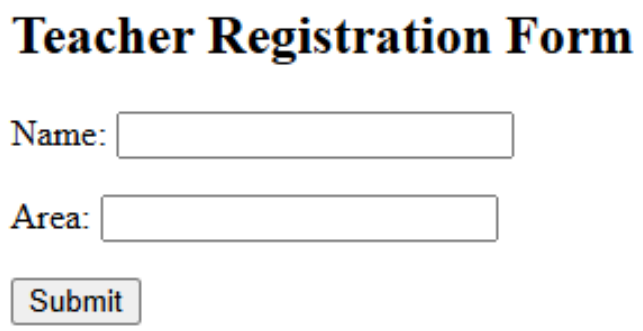
```
        if form.is_valid():
            form.save()
            return redirect("index")

    else:
        form = InputForm()

    return render(request, "MyApp1/input.html", {"form": form})
```

To add a new teacher to the database, I used the lines `if form.is_valid(): form.save()`. This code checks whether the information entered by the user is valid and then saves it into the database. Once the data is stored, it can be displayed on the website. In summary, the user enters a teacher's **name** and **area** into the form, the data is saved into the database, and it can then be displayed on the webpage.

While developing the form in Django, I encountered a common issue. As mentioned earlier, Django requires several Python files to be edited in order to make a new page accessible. Even after adding the correct code to the URL file, the form did not appear on the website. To fix this, I created a new HTML file named input.html. Inside this file, I added the form and used the line `{{ form.as_p }}`. This command automatically converts the Django form into HTML paragraphs, allowing it to appear on the webpage without manually writing the full form structure.

However, the form still could not interact with the database at first. This was because I had not included `{% csrf_token %}`. In Django, this token is required for security reasons. Without it, Django blocks the form from being submitted. After adding the token, the form was able to successfully submit data.

Since I used `{{ form.as_p }}`, Django automatically handles the display of the form elements in the HTML page. Once the form submission is completed, the new teacher information is saved in the database. The website also includes a link that opens a new page where users can add new teachers. After submitting the form, the user is returned to the home page, where the updated teacher information is displayed.




## What have I learned?

Through this project I learned how to create user input forms in Django, as well as how to create and link different pages within a Django website and use Python inside HTML templates. Creating input forms is an important feature, especially for websites like the BSSS Unit Outline generator, because forms are used to collect and organise information from users. Learning how to build forms that are simple and efficient can make the website much easier and faster for people to use.

I also learned how to create and connect multiple pages in Django. This process can be a bit confusing at first because you have to edit several different Python files, such as the views and URL files, just to make one page work. However, it is a

useful skill because most websites have many different pages that link together. Knowing how to set this up will help if I need to expand a website or add more features in the future.

Another thing I learned was how to include Python directly in HTML using Django's template system. This is helpful because it saves time and keeps the code more organised, especially when displaying information from the database. One challenge for me was that HTML is easier for me to work with than Python, so sometimes writing Python inside HTML took a bit longer. Even so, it's a really useful feature that makes websites more dynamic and functional.

# Log 3

## Feature definition

Adding a button to delete teachers from the database. This feature is important in case there are teachers that leave, or if there is any misinformation in the database.

## Solution process

To build on the form I created earlier, I wanted to add a feature that lets users remove teachers from the database. At first, I thought it would only require some HTML and a new view. I started by adding code to the 

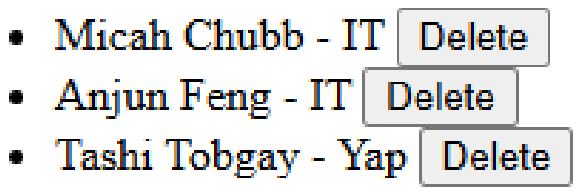
. By creating the function `def delete_teacher(request, teacher_id):`, I was able to target a specific teacher using their ID. Django automatically assigns an ID to each entry, so I used that to find the correct teacher and delete it with `t.delete()`. After that, I used `return redirect("index")` to reload the homepage so the updated list would show.

Next, I worked on the HTML. I added a simple delete button next to each teacher's name by creating a list using `{{ t.Name }} - {{ t.Area }}` and then adding a tag with a button. This made a delete button appear next to every teacher. When clicked, it sends a request that runs something like `delete_teacher(request, teacher_id=3)`.

One issue I ran into was an error from Django because I forgot to update the URLs. Without a URL, Django doesn't know what to do when the button is clicked. To fix this, I added the line:

```
path('delete/<int:teacher_id>/', MyApp1.views.delete_teacher,
name='delete_teacher'),
```

This tells Django to run the delete function using the teacher's ID when the button is pressed. After fixing that, the delete feature worked properly and removed teachers from the database as expected.




### What have I learned?

Through this process I learned how different parts of a DJango project connect, especially how views, URLs, and templates work together. I also learned how to handle user input using forms and store data in a database, as well as import data using CSV files. One thing that stood out is how useful Django templates are for mixing python with HTML to display dynamic content. In the future, I could apply these skills to build more complex websites, especially ones that require user input, data storage or multiple pages that are linked together.

## Bibliography (brief explanation of each link plz)


- *Django documentation | Django documentation | Django*. (n.d.). Docs.djangoproject.com. https://docs.djangoproject.com/

This was the main source I used to understand how Django works. It helped me learn about views, URLs, templates, and forms, and I referred to it when I got stuck on errors.

- *Django Web Framework (Python)*. (2019, November 30). MDN Web Docs. https://developer.mozilla.org/en-US/docs/Learn/Server-side/Django

This guide explained Django in a more beginner-friendly way. It was useful for understanding how different parts of a web app connect, especially when linking HTML and Python.

- Stack Overflow. (2024). *Stack Overflow - Where Developers Learn, Share, & Build Careers*. Stack Overflow. https://stackoverflow.com/

I used this when I ran into specific errors. Reading other people's questions and answers helped me fix problems, especially with forms and URLs.

- W3Schools. (2024). *HTML Tutorial*. W3schools.com. https://www.w3schools.com/html/

I used this to refresh my HTML knowledge. It helped with things like forms, buttons, and linking elements, which I needed when building my pages.

# Demonstration Video

https://docs.google.com/videos/d/1b-EWct3FyTMH7ZMKIPE4vZOO0tyL8sQBa3oMxdqiXEs/edit?usp=sharing

# Code

*https://github.com/8001688-cmyk/PracticeSchool1.git*

# Interview Notes